

Слайд 1 (титульный; таблица терминов — справочный материал ко всей лекции)

Лекция 3. Хранение в базах данных (часть I)

0. Термины

Ниже приведены основные термины из лекции, относящиеся к хранению данных в диск-ориентированных СУБД. Для каждого термина дано краткое пояснение на русском языке и пример из материала лекции.

Термин (English)	Пояснение	Пример
Disk-oriented DBMS (диск-ориентированная СУБД)	СУБД, в которой основное постоянное хранилище базы находится на энергонезависимом диске, а обработка данных выполняется после загрузки нужных страниц в оперативную память.	PostgreSQL, MySQL при стандартной конфигурации.
Volatile storage (энергозависимая память)	Память, содержимое которой теряется при отключении питания. Поддерживает быстрый произвольный доступ с байтовой адресацией.	DRAM.
Non-volatile storage (энергонезависимая память)	Хранилище, сохраняющее данные после выключения питания. Доступ блочный (нельзя обратиться к отдельному байту без загрузки всего блока).	SSD, HDD, сетевое хранилище типа Amazon S3.
Block-addressable (блочная адресация)	Способ доступа, при котором минимальная единица чтения/записи — блок (обычно 4 КБ).	Аппаратная страница в 4 КБ — гарантируется атомарная запись только одного такого блока.
Sequential access / Random access (последовательный / случайный доступ)	Последовательный доступ — чтение непрерывного диапазона данных; случайный — переход в произвольные места. На реальных устройствах последовательный значительно быстрее.	Random I/O на SSD ≈ 100 мкс, Sequential I/O может давать на порядки большую пропускную способность.
Storage Manager (менеджер хранения)	Компонент СУБД, управляющий файлами базы данных на диске, представляющий их как набор страниц. Отвечает за выделение места, чтение и запись страниц.	В MySQL InnoDB — это слой управления табличным пространством и страницами.
Page (страница)	Фиксированный блок данных на диске, базовая единица ввода-вывода СУБД. Различают аппаратную страницу (обычно 4 КБ), страницу ОС и страницу БД.	Страница PostgreSQL — 8 КБ, MySQL InnoDB — по умолчанию 16 КБ.

Термин (English)	Пояснение	Пример
Page ID (идентификатор страницы)	Уникальный номер, идентифицирующий конкретную страницу в базе данных. Может быть смещением в файле или логическим идентификатором, разрешаемым через директорию страниц.	В PostgreSQL page ID — составная часть ctid , например (номер страницы, номер слота).
Heap file	Неупорядоченный набор страниц, в котором кортежи размещаются в произвольном порядке на любом подходящем свободном месте.	Таблица в PostgreSQL по умолчанию хранится как куча.
Page Directory	Метаданные, отображающие логический page ID в физическое расположение (файл и смещение). Хранит также информацию о свободном месте и типах страниц.	В InnoDB для каждого табличного пространства есть своя структура, аналогичная директории.
Slotted page	Макет страницы, при котором в начале находится массив слотов (смещений), указывающих на кортежи, хранящиеся с конца страницы. Массив растёт вперёд, данные — назад. Позволяет перемещать кортежи внутри страницы, не меняя внешние индексы.	Каждая страница данных в PostgreSQL организована как slotted page.
Slot array (массив слотов)	Фиксированного размера массив в заголовке slotted page, хранящий смещения начала каждого кортежа внутри страницы.	Если в странице 3 кортежа, slot array содержит три смещения, указывающих на их начало.
Tuple (кортеж)	Запись в реляционной базе данных, строка таблицы. Представляет собой последовательность байтов с заголовком и данными атрибутов.	Строка таблицы students(id INT, name VARCHAR) — один кортеж.
Tuple header (заголовок кортежа)	Метаданные внутри кортежа: битовая карта NULL-значений, информация о видимости транзакциями, возможно другие служебные поля.	Битовая карта длиной 5 бит для таблицы с 5 столбцами в заголовке каждого кортежа.
Record ID / Tuple ID (идентификатор записи)	Уникальный идентификатор физического местоположения кортежа, обычно состоит из номера файла, номера страницы и номера слота. Неизменяем в течение жизни кортежа на одной странице, но может измениться при перемещении.	ctid в PostgreSQL: (0, 1) — страница 0, слот 1. SQLite использует rowid как скрытый первичный ключ.

Термин (English)	Пояснение	Пример
Overflow page (страница переполнения)	Отдельная страница, используемая для хранения больших значений атрибутов, которые не помещаются в основную страницу кортежа. В кортеже хранится ссылка на такую страницу.	В PostgreSQL атрибут длиннее 2 КБ (при 8-КБ странице) автоматически выносится в TOAST-таблицу (overflow).
External value storage (внешнее хранилище значений)	Хранение больших объектов (BLOB) вне файлов базы данных. В кортеже хранится только ссылка (путь/URI). СУБД может прочитать файл, но не управляет транзакционной целостностью этого файла.	Oracle B-файлы, Microsoft FileStream. Рекомендация: объекты > 128 КБ выносить во внешнее хранилище.
Padding / Alignment (заполнитель / выравнивание)	Добавление пустых байтов после атрибута для размещения следующего атрибута по границе машинного слова (обычно 64 бита). Обеспечивает корректный и быстрый доступ к полям.	После 32-битного целого добавляются 32 нулевых бита, чтобы следующий 64-битный timestamp начинался с адреса, кратного 8.

Слайд 2 — Введение и архитектура системы

1. Введение и архитектура системы

Мы переходим от логического уровня SQL и реляционной модели к тому, как реально построить программную систему, выполняющую запросы и хранящую данные. Рассматриваем классическую архитектуру СУБД, зародившуюся в 1970-х, ориентированную на хранение по строкам (row-oriented storage). Основное предположение — disk-oriented (диск-ориентированная) архитектура: первичное хранилище базы данных находится на энергонезависимом диске (non-volatile disk), будь то SSD, HDD или сетевое хранилище. По классической фон-неймановской схеме, данные для обработки необходимо загружать с диска в энергозависимую память (volatile storage, DRAM). СУБД отвечает за последующую запись изменений обратно на энергонезависимый диск.

Слайд 3 — Слои архитектуры СУБД (снизу вверх)

База данных представляется как набор слоёв с чётко определёнными API. Снизу вверх:

- **Disk Manager** (менеджер диска) — читает и записывает данные с диска.
- **Buffer Manager** (менеджер буферов) — загружает страницы с диска в память и предоставляет доступ к ним другим компонентам.
- **Access Methods** (методы доступа) — осуществляют доступ к данным для операторов выполнения запросов.
- **Query Planner/Optimizer** (планировщик/оптимизатор запросов) — преобразует SQL-запросы в физические планы выполнения. На самом верху — приложение, взаимодействующее с фронтендом планирования запросов.

В этой лекции мы начинаем с самого низа — уровня хранения.

Слайд 4 — Иерархия хранения

2. Иерархия хранения и время доступа

Иерархия хранения (storage hierarchy) выглядит классически: чем ближе к CPU, тем быстрее, меньше и дороже память; чем дальше — тем больше, медленнее и дешевле в расчёте на гигабайт. Типичная лестница уровней: регистры CPU → кэши CPU → DRAM → SSD → HDD → сетевое хранилище. Ключевая демаркационная линия разделяет волатильные (volatile) и неволатильные (non-volatile) устройства.

Волатильные устройства (CPU регистры, кэши, DRAM):

- Теряют состояние при отключении питания.
- Поддерживают произвольный доступ с байтовой адресацией (byte-addressable), можно перейти к любому байтовому смещению без чтения лишних данных.
- Для целей курса называем это "память" (memory).

Неволатильные устройства (SSD, HDD, сетевое хранилище типа Amazon S3):

- Сохраняют данные после потери питания.
- Являются блочно-адресуемыми (block-addressable). Минимальная единица доступа — блок (обычно 4 КБ). Чтобы прочитать один байт, нужно загрузить целый блок/страницу.
- Называем это "диск" (disk).
- Важное отличие: последовательный доступ (sequential access) значительно быстрее случайного (random access).

Слайд 5 — Время доступа к хранилищам

Для осознания порядка различий в задержках полезно пересчитать их в «человеческий» масштаб — приняв чтение из L1-кэша за 1 секунду:

Уровень	Реальная задержка	В масштабе «L1 = 1 секунда»
L1 cache	~1 ns	1 секунда
L2 cache	~4 ns	4 секунды
DRAM	~100 ns	~1,5 минуты
SSD	~16 000 ns	~4,4 часа
HDD	~2 000 000 ns	~3,3 недели
Сетевое хранилище (S3)	~50 000 000 ns	~1,5 года
Магнитная лента	~1 000 000 000 ns	~31,7 года

Ленточные накопители поэтому используют только для архивов (например, Amazon Glacier): задержка первого байта там измеряется секундами и минутами.

Слайд 6 — Последовательный и случайный доступ

Последовательный и случайный доступ: На реальном оборудовании разница огромна. Случайное чтение блока даже на быстром SSD занимает около 80–100 микросекунд, а последовательное чтение

смежных блоков идёт на полной пропускной способности устройства (сотни мегабайт — гигабайты в секунду), поэтому в пересчёте на байт оно на порядки дешевле. СУБД стремится максимизировать последовательный доступ. Например, MySQL при записи "грязных" страниц на диск сначала пишет их последовательно в double write buffer, а затем в фоне выполняет случайные записи в файлы данных.

Слайд 7 — Цели проектирования диск-ориентированной СУБД

3. Цели проектирования диск-ориентированной СУБД

Главная цель — позволить СУБД управлять базой данных, превышающей объём доступной памяти. Если доступно 8 ГБ ОЗУ, мы должны работать с БД размером и 16 ГБ, и 32 ГБ, и больше. Задача — обеспечить иллюзию наличия достаточной памяти за счёт интеллектуального перемещения данных между диском и памятью.

Почему это нетривиально, видно из трёх ограничений, с которыми борется дизайн любой диск-ориентированной СУБД:

- Пользователи хотят обрабатывать объёмы данных, кратно превышающие доступную RAM.
- Чтение и запись на диск — дорогие операции (вспомним масштаб задержек из раздела 2).
- Случайный доступ к диску существенно медленнее последовательного.

Поэтому необходимо избегать длительных остановок при ожидании I/O, эффективно переиспользовать загруженные в память данные, а также применять оптимизации, минимизирующие обращения к диску. Если рабочий набор (working set) всех активных запросов превышает объём памяти, проблема неизбежна, но в общем случае она решается.

Слайд 8 — Принцип устройства памяти диск-ориентированных СУБД

4. Дисковые СУБД

Соберём картину целиком. На самом нижнем уровне находится диск, хранящий набор файлов, разбитых на блоки фиксированного размера — **страницы**. У каждой страницы есть заголовок с метаданными, у каждого файла, содержащего N страниц, — свой идентификатор.

Уровнем выше расположена оперативная память с пулом буферов (buffer pool): в него по требованию загружаются страницы, включая служебные страницы с метаданными. Когда движок исполнения запрашивает нужную страницу по её указателю, она быстро доставляется в память — тем самым диск и память работают как единая система. Как именно это делается, мы разберём в следующей лекции, посвящённой Buffer Manager.

Над уровнем хранения находится движок исполнения запросов, который читает и изменяет содержимое страниц.

Слайд 9 — Файловое хранилище

5. Файловое хранение (File Storage)

На низком уровне база данных — это файлы на диске. Некоторые системы хранят БД в одном файле (например, SQLite, DuckDB), большинство — в нескольких. С точки зрения ОС это обычные файлы, созданные через `fopen/fwrite`.

Традиционно форматы файлов проприетарны: файл Postgres нельзя открыть в MySQL или Oracle, так как никто иной не знает, как интерпретировать байты. Новый тренд — переносимые форматы (portable file formats, например Parquet), имеющие открытую спецификацию и доступные разным системам.

Слайд 10 — Файловые системы и Storage Manager

СУБД обычно работают на стандартных файловых системах (ext4, NTFS и т.п.). В 1980-х некоторые системы создавали собственные ФС поверх "сырых" блочных устройств. Сейчас это редкость, встречается в высококлассных enterprise-системах вроде Oracle ASM, где СУБД внедряет свою ФС в ОС. Выигрыш производительности порядка 15% не оправдывает инженерных усилий для большинства систем.

Слайд 11 — Storage Manager

6. Storage manager

Компонент, отвечающий за управление файлами БД, — **Storage Manager** (менеджер хранения, иногда его называют Disk Manager). Его обязанности:

- представлять файлы базы данных как коллекцию страниц фиксированного размера;
- поддерживать файлы в согласованном состоянии: отслеживать, какие данные в какие страницы записаны и где осталось свободное место;
- самостоятельно планировать порядок чтения и записи страниц, переопределяя его ради производительности, а не полагаясь на решения ОС.

За репликацию и резервные копии данных storage manager обычно не отвечает: это делается либо ниже — на уровне файловой системы и RAID, либо выше — компонентами, работающими с логическими копиями кортежей.

Слайд 11 — Storage Manager (отдельного слайда про mmap в колоде нет; блок относится к тезису «не полагаться на ОС» слайда 11)

7. СУБД против операционной системы

Высокоуровневая задача СУБД — поддержка БД больше памяти — напоминает виртуальную память. Можно было бы использовать **mmap** для отображения файла в адресное пространство процесса, чтобы ОС управляла перемещением страниц. Однако при страничном промахе (page fault) процесс блокируется. Для СУБД, особенно с записью, использовать **mmap нельзя**. СУБД почти всегда хочет контролировать всё сама, так как обладает большей информацией о данных и запросах, чем ОС, и может делать это эффективнее. ОС — не друг СУБД.

Можно применять отдельные механизмы ОС:

- **madvise** — подсказать ОС о планируемом чтении определённых страниц.
- **mlock** — запретить выгрузку диапазонов памяти в своп.
- **msync** — принудительно сбросить диапазоны памяти на диск.

Но реализация собственных процедур СУБД даёт лучший контроль и производительность.

Слайд 12 — Страницы базы данных

8. Страницы базы данных (Database Pages)

СУБД организует базу данных в файлах как блоки фиксированного размера — **pages** (страницы).

Страницы содержат разные виды данных: кортежи (tuples), индексы, метаданные, директории.

Большинство систем не смешивают типы страниц внутри одного файла. Некоторые требуют, чтобы вся информация для интерпретации страницы хранилась в ней самой (например, Oracle).

Каждая страница имеет уникальный идентификатор — **page ID** (идентификатор страницы). Если БД — один файл, page ID может быть смещением в файле. Часто page ID отображается в путь к файлу и смещение. Page ID может быть уникальным в рамках экземпляра СУБД, базы данных или таблицы.

Слайд 13 — Три понятия страниц в СУБД

Три понятия страницы:

1. **Аппаратная страница (hardware page)**: обычно 4 КБ. Это максимальный размер атомарной записи, гарантируемый устройством хранения (либо все 4 КБ запишутся, либо ни одного).
2. **Страница ОС (OS page)**: в Linux по умолчанию 4 КБ, бывают huge pages (2 МБ – 1 ГБ в x64-системах).
3. **Страница БД (database page)**: 512 байт – 32 КБ (в SQLite по умолчанию 4 КБ, в Postgres и SQL Server 8 КБ по умолчанию, в MySQL до 16 КБ).

Слайд 14 — Выбор размера страницы БД

Размер страницы БД зависит от окружения, характера данных и нагрузки.

- Для read-heavy (много чтения) — большие страницы (вплоть до 1 МБ), так как последовательные сканирования читают много непрерывных данных, и каждая загруженная страница содержит полезные кортежи.
- Для write-heavy (много обновлений/вставок/удалений) — небольшие страницы (4–16 КБ), потому что при изменении даже одного байта нужно переписать всю страницу на диск (атомарная запись), а большая страница означает больший объём записи.

Если страница БД больше аппаратной, СУБД должна предпринимать дополнительные меры для безопасной записи, так как система может упасть в процессе записи страницы.

Слайд 15 — Heap File (Куча)

9. Организация файлов: Heap File (Куча)

Существует несколько способов организации страниц в файлах: древовидная (tree), ISAM

(последовательная, устарела), хешированная (hashing). Наиболее распространена **Heap File**

Organization (неупорядоченная куча) — неупорядоченный набор страниц, где кортежи хранятся в случайном порядке.

Поскольку SQL-модель не упорядочена, кортежи можно размещать где угодно, где есть свободное место. API heap-файла предоставляет операции создания страниц, итерации по всем страницам (для последовательного сканирования) и получения отдельной страницы по ID.

Важное следствие: у кучи нет внутренней структуры порядка, поэтому сама по себе она не ускоряет поиск по ключу. Для быстрого доступа поверх кучи создают отдельные индексы, которые ссылаются на

кортежи через Record ID (комбинацию номера страницы и слота — о нём поговорим в разделе 11).

Слайд 16 — Heap File (Куча): API и директория страниц

Для поиска страницы по её ID часто используется **Page Directory** (директория страниц) — специальные страницы с метаданными, содержащие записи для каждого логического объекта БД (страницы данных, индексов) и информацию о свободном месте. Директория **должна** синхронизироваться с актуальными страницами, но допускает некоторую рассинхронизацию на диске (полное восстановление возможно, но дороже). Также директория отслеживает **свободное пространство** на страницах, список **полностью пустых страниц** и **типы страниц**.

Слайд 17 — Макет страницы

10. Макет страницы (Page Layout)

Каждая страница имеет **заголовок (header)** с метаданными:

- Размер страницы
- Контрольная сумма (checksum)
- Версия СУБД (для совместимости при обновлениях)
- Информация о видимости транзакций (transaction visibility)
- В некоторых системах — схема таблицы (для самодостаточности)
- Суммарная информация / эскизы (sketches) значений для ускорения поиска без сканирования всей страницы.

Слайд 18 — Три подхода к размещению данных внутри страниц (включая Slotted Pages)

Существует три основных подхода к размещению данных внутри страниц:

1. **Tuple-oriented** (кортеж-ориентированное) — кортеж целиком хранится непрерывно в странице.
2. **Log-structured** (лог-структурированное) — хранятся дельты изменений кортежа, а не сам кортеж.
3. **Index-organized** (индекс-ориентированное) — данные хранятся в структуре дерева.

В этой лекции фокус на tuple-oriented подходе.

Slotted Pages (страницы со слотами)

Наивный подход — отслеживать количество кортежей и добавлять новые последовательно в конец. Проблемы возникают при удалении кортежей или наличии атрибутов переменной длины: сложно утилизировать освободившееся место и делать уплотнение (compaction), так как изменится физический адрес кортежа, требуя обновления индексов.

Решение — **slotted pages**. В начале страницы находится заголовок, затем **slot array** (массив слотов), хранящий смещения кортежей внутри страницы. Сами данные кортежей размещаются с конца страницы, растут навстречу массиву слотов. Страница заполнена, когда они встречаются.

Слайд 19 — Преимущества Slotted Pages

Преимущества:

- При удалении кортежа и уплотнении достаточно обновить запись в slot array, не изменяя внешние индексы. Изменения ограничены одной страницей.

- Операции уплотнения в памяти дешёвы по сравнению с операциями чтения с диска.

Слайд 20 — Идентификаторы записей (Record ID / Tuple ID)

11. Идентификаторы записей (Record IDs)

Каждому логическому кортежу назначается уникальный идентификатор, представляющий его физическое местоположение — **Record ID** (иногда Tuple ID, Row ID).

Обычно это комбинация: file ID, page ID и номер слота (slot number) в странице.

Большинство СУБД не хранят Record ID внутри кортежа, а вычисляют его во время выполнения.

Размер варьируется: 4 байта в Ingres, 6 байт (page ID + slot) в Postgres (ctid). SQLite хранит row ID всегда и использует его как первичный ключ, если тот не задан. Приложение **не должно** полагаться на Record ID, так как они могут меняться при перемещении данных (compaction, реорганизация), что допустимо в неупорядоченной реляционной модели.

Слайд 21 — Макет кортежа

12. Макет кортежа (Tuple Layout)

Кортеж — это последовательность байтов (не обязательно непрерывных). СУБД интерпретирует эти байты как значения атрибутов.

Каждый кортеж начинается с заголовка. **Заголовок кортежа (Tuple Header)** содержит:

- Информацию о видимости для протокола управления конкурентным доступом (какая транзакция создала/модифицировала).
- Битовую карту (bit map) NULL-значений.
- Схема БД в заголовке не дублируется, она хранится в каталоге СУБД; на уровне страницы все кортежи принадлежат одной таблице.

В PostgreSQL информация о конкурентном доступе реализована на физическом уровне через служебные поля **xmin** и **xmax**, которые лежат в основе механизма MVCC (Multi-Version Concurrency Control — управление конкурентным доступом с многоверсионностью). Поле **xmin** хранит идентификатор транзакции, которая создала (вставила) текущую версию кортежа, а **xmax** — идентификатор транзакции, которая удалила или изменила его (создав новую версию строки). Благодаря такой физической разметке, СУБД может определять, какая именно версия кортежа видна конкретной транзакции в данный момент, что позволяет читающим транзакциям не блокировать пишущие и наоборот.

Данные кортежа (Tuple Data):

- Атрибуты обычно хранятся в порядке, заданном при **CREATE TABLE**.
- Должны быть выровнены (word aligned), обычно по 64-битной границе, чтобы CPU мог прочитать машинное слово за одну операцию: невыровненный доступ на одних архитектурах работает медленнее, на других — приводит к аппаратной ошибке.

Слайд 22 — Проблемы выравнивания (Alignment) и заполнители (Padding)

Проблемы выравнивания (Alignment) и заполнители (Padding)

Если схема содержит поля разного размера, могут возникать переходы через границы машинных слов. Например, 32-битное целое, затем 64-битная временная метка, затем 16-битный символ. Без выравнивания доступ к невыровненным данным замедляется или вызывает ошибки на некоторых архитектурах. Решение — добавление **padding** (заполнителей) после атрибутов для выравнивания следующего атрибута по нужной границе.

Альтернатива — физически переупорядочить столбцы для более плотной упаковки (в исследовательских работах встречается, но коммерческие СУБД так обычно не делают). Как правило, используется padding.

12.1. Практика: структура файлов и страниц в PostgreSQL

Слайд 23 — Файлы таблицы в PostgreSQL (*новый слайд, добавлен в колоду*)

```
-- Путь к основному файлу таблицы (relfilenode)
SELECT pg_relation_filepath('demo.bookings.flights');

-- Размер таблицы в байтах и в страницах (по 8 КБ)
SELECT pg_relation_size('demo.bookings.flights') AS size_bytes,
       pg_relation_size('demo.bookings.flights') / 8192 AS size_pages;

-- Информация о файле на диске (размер, время изменения)
SELECT (pg_stat_file(pg_relation_filepath('demo.bookings.flights'))).*;
```

Слайд 24 — Slotted Pages в PostgreSQL: заголовок страницы и массив слотов (*новый слайд*)

Страница heap в PostgreSQL имеет заголовок, за которым следует массив слотов (line pointers), а данные кортежей растут с конца страницы навстречу. Функция `page_header` показывает границы (lower — конец массива слотов, upper — начало данных кортежей), а `heap_page_items` — содержимое каждого слота.

```
-- Заголовок страницы: lower — смещение конца массива слотов,
-- upper — смещение начала области данных кортежей.
-- Когда lower == upper, страница заполнена.
SELECT * FROM page_header(get_raw_page('demo.bookings.flights', 0));

-- Все line pointers (слоты) на странице 0.
-- lp      — номер слота (начинается с 1)
-- lp_off   — смещение данных кортежа внутри страницы
-- lp_flags — 0 = unused, 1 = normal, 2 = redirect, 3 = dead
-- lp_len   — длина кортежа в байтах
SELECT lp, lp_off, lp_flags, lp_len
FROM heap_page_items(get_raw_page('demo.bookings.flights', 0))
ORDER BY lp;
```

Комментарий: массив слотов растёт от начала страницы (после заголовка), а кортежи — от конца. `lp_off` показывает, где именно в странице лежат данные каждого кортежа.

Слайд 25 — Заголовок кортежа и разбор данных в PostgreSQL (новый слайд)

```
-- Полный заголовок кортежа + сырые данные для страницы 0.
-- t_xmin, t_xmax — идентификаторы транзакций (MVCC)
-- t_field3      — cmin/cmax (упакованы)
-- t_ctid       — текущий ctid этой версии кортежа
-- t_infomask    — битовые флаги (видимость, наличие NULL и т.д.)
-- t_infomask2   — дополнительная информация (число атрибутов и пр.)
-- t_hoff       — смещение начала данных от начала кортежа
-- t_bits       — битовая карта NULL-значений
SELECT lp, t_xmin, t_xmax, t_field3, t_ctid,
       t_infomask, t_infomask2, t_hoff, t_bits, t_data
FROM heap_page_items(get_raw_page('demo.bookings.flights', 0))
ORDER BY lp;

-- Разбор данных кортежа по атрибутам (как их видит СУБД)
SELECT lp, t_ctid,
       tuple_data_split('demo.bookings.flights'::regclass,
                       t_data, t_infomask2, t_bits) AS attributes
FROM heap_page_items(get_raw_page('demo.bookings.flights', 0))
ORDER BY lp;
```

Слайд 26 — MVCC на практике: версии строк на странице (новый слайд)

В PostgreSQL каждая версия строки — это отдельный физический кортеж. xmin хранит ID транзакции, создавшей версию, xmax — ID транзакции, удалившей или изменившей её. Чтобы увидеть, как это работает, можно выполнить простой сценарий: вставить строку, обновить её и посмотреть обе версии на странице.

```
-- Смотрим физическую страницу: будут видны обе версии
SELECT lp, t_xmin, t_xmax, t_ctid, t_data
FROM heap_page_items(get_raw_page('demo.bookings.flights', 0))
WHERE lp IN (1, 2) -- номера слотов из предыдущих шагов
ORDER BY lp;
```

Слайд 27 — ctid — аналог Record ID в PostgreSQL (новый слайд)

В PostgreSQL аналог Record ID — системный столбец ctid, состоящий из номера страницы (block) и номера слота (tuple index).

```
-- ctid для каждой строки: (номер страницы, номер слота)
SELECT ctid, * FROM demo.bookings.flights LIMIT 10;

-- Разбор ctid на составляющие через системные функции
SELECT ctid,
       (ctid::text::point)[0]::int AS page_number,
       (ctid::text::point)[1]::int AS slot_number
```

```
FROM demo.bookings.flights  
LIMIT 10;
```

Слайд 28 — Целочисленные типы (INTEGER, BIGINT, SMALLINT)

13. Представление данных (Data Representation)

Целочисленные типы (INTEGER, BIGINT, SMALLINT, TINYINT)

Хранятся в нативном формате C/C++. Необходимо учитывать порядок байтов (endianness). SQLite с его динамической типизацией — отдельный случай: тип каждого значения помечается тегом (serial type) в заголовке записи, а целые числа хранятся на диске как big-endian целые переменной длины, что снимает проблему порядка байтов и экономит место на маленьких значениях. Большинство же систем хранят целые в нативном представлении с дополнительными метаданными.

Слайд 29 — Числа с плавающей и фиксированной точкой

Числа с плавающей точкой и фиксированной точностью

- **FLOAT / REAL** — типы с переменной точностью. Используют стандарт IEEE-754, реализованный аппаратно. Быстры благодаря инструкциям CPU, но не гарантируют точного представления десятичных дробей. Пример: $0.1 + 0.2$ может дать 0.30000000000000004 , а не ровно 0.3 , из-за особенностей двоичного представления.
- **NUMERIC / DECIMAL** — типы с фиксированной точностью. Позволяют задавать произвольную точность и масштаб. Реализуются программно: хранится дополнительная метainформация о положении десятичной точки и масштабе, а значение хранится как массив байтов (по сути, строка или упакованное представление). Это гарантирует точность вычислений, критичную в финансах, банковском деле, научных расчётах. Пример реализации: в Postgres тип `numeric` — это структура с набором служебных полей (знак, масштаб, положение точки) и массив байтов. Сложение двух `numeric` (код из Postgres) содержит множество проверок (положительное/отрицательное, бесконечность, NaN) и выполняется программно, что на порядки медленнее аппаратного сложения, но обеспечивает ожидаемую точность.

Слайд 30 — Строки и бинарные данные (VARCHAR, TEXT, BLOB)

Строки и бинарные данные (VARCHAR, VARBINARY, TEXT, BLOB)

- Если значение невелико, хранится прямо в кортеже (inline): длина, затем байты данных.
- Если значение слишком велико (обычно больше половины размера страницы или, как в Postgres, больше 2 КБ при 8-КБ странице), оно выносится в **overflow pages** (страницы переполнения). В кортеже хранится указатель (record ID) на overflow-страницу и размер данных. Если overflow-страница переполняется, указатели могут образовывать цепочку.

Механизм **TOAST в PostgreSQL** — это и есть реализация overflow pages: значения длиннее ~2 КБ автоматически сжимаются и при необходимости выносятся из основной страницы в служебную TOAST-таблицу.

Оптимизации для overflow pages:

- Сжатие overflow-страниц (snappy, gzip).
- **Prefix truncation** (усечение с префиксом) или **German strings**: в кортеже хранится не только указатель и размер, но и префикс значения. Это позволяет отсеять неподходящие кортежи при строковом поиске (например, **LIKE 'Andy%'**), не обращаясь к overflow-странице.

External Value Storage (внешнее хранилище значений): Некоторые СУБД позволяют хранить большие значения (например, 20 ГБ фильм) в файлах вне БД, а в кортеже хранить URI или указатель. СУБД может прочитать этот файл по требованию, но **не гарантирует** транзакционную безопасность (durability) и другие гарантии для этих данных. Oracle называет это B-Files, Microsoft — FileStream. (TOAST в PostgreSQL к внешнему хранилищу **не относится**: данные остаются внутри БД, в служебной TOAST-таблице, и полностью транзакционны.) Рекомендация (старое исследование Джима Грея): всё, что больше 128 КБ, хранить снаружи; меньше — внутри. Так экономится место на дорогом оборудовании СУБД.

Слайд 30 — Строки и бинарные данные (продолжение: External Value Storage)

"Хранение персональных данных (по 152-ФЗ) часто требует шифрования. Физическое вынесение BLOB-объектов (например, сканов паспортов) вне базы данных позволяет делегировать шифрование файловой системе, оставляя в кортеже только ссылку, что также ускоряет сканирование таблицы".

Слайд 31 — Дата и время (TIME, DATE, TIMESTAMP, INTERVAL)

Дата и время (TIME, DATE, TIMESTAMP, INTERVAL)

Обычно хранятся как 32- или 64-битное целое — количество микросекунд или миллисекунд от внутренней точки отсчёта (в PostgreSQL **timestamp** — 64-битное число микросекунд с 1 января 2000 года; в Unix-подобных системах часто от Unix epoch, 1 января 1970). Некоторые системы имеют свои внутренние форматы и поддержку часовых поясов.

Слайд 32 — Представление NULL-значений: три подхода

Представление NULL-значений

Три подхода:

1. **Null Column Bitmap Header** — наиболее распространён в row-stores (построчных хранилищах). В заголовке кортежа битовая карта, где каждый бит соответствует атрибуту и показывает, является ли он NULL.
2. **Special Values (специальные значения)** — выделяется специальное значение домена для обозначения NULL (например, **INT32_MIN**). Часто применяется в column-stores (колоночных хранилищах), так как экономит метаданные кортежа.
3. **Per-Attribute Null Flag (флаг на атрибут)** — перед каждым значением хранится флаг NULL. Крайне нежелателен, так как требует выравнивания (флаг в 1 бит превращается в целый байт или более), раздувает размер. Известна лишь одна система, сделавшая так и быстро отказавшаяся. **Не делайте так.**

Слайд 32 (заключительные замечания; отдельного слайда с заключением в колоде нет)

Заключение

В этой лекции мы разобрали основы хранения данных: диск-ориентированная архитектура, иерархия памяти, организация файлов, страницы, slotted pages, кортежи и представление различных типов данных, включая NULL и большие значения. Мы движемся снизу вверх по стеку СУБД. Следующая лекция будет посвящена **Buffer Manager** (менеджеру буферов): как загружать страницы в память, управлять ими, передавать другим частям системы и безопасно сбрасывать изменения на диск.